

Three Fields Arrive, One Does Not Exist

The blocker the last report closed on is fixed as far as Afrinext's own data allows. The payer's number and country now reach the provider from the buyer's own account. A real iPayMoney charge still refuses — and it now refuses naming `customer_name`, which is a field no part of Afrinext has ever collected.

Branch `claude/create-app-repository-1cxsih`HEAD `49c043f`Unit tests **491 passed**Browser tests **31 passed**Mutations killed **24 / 26**Real sandbox calls **0**

THE ONE-PARAGRAPH ANSWER

iPayMoney requires four things Afrinext was not sending. **Three now arrive:** the payer's verified phone, the buyer's country when we hold one, and the payment channel when the caller chose one. **The fourth — `customer_name` — cannot be sent, because Afrinext has never asked a buyer for a name.** What sits in the name columns are placeholders written at signup to satisfy a NOT NULL: a synthetic address and the phone string.

Sending either would be inventing a value for a provider-required field, so the adapter still refuses. That refusal is the correct outcome and the honest end of this milestone. Closing the gap is a product decision — **decision 1 in section L.**

A

Commit and scope

One commit, `49c043f`, on `claude/create-app-repository-1cxsih`. Not merged to `main`.

FILE	Δ	WHY
packages/core/src/orders/payment.ts	+166 -10	The wiring: a buyer-identity loader, the fields it contributes to <code>ChargeInput</code> , and the classification of a throw from <code>createCharge</code>
.../orders/checkout-provider-wiring.test.ts	+415 new	13 tests driving the real <code>createCheckout → initiatePayment</code> path against a real database
packages/core/src/test/harness.ts	+34 -3	<code>createTestUser</code> can now create the Better Auth credential row that phone-OTP signup creates, so tests exercise the same join production uses
apps/web/e2e/checkout.spec.ts	+16 -1	Asserts a real OTP signup produces the identity link a charge depends on
docs/providers/ipaymoney/README.md	+18	Assumption A11, and what the checkout path supplies

NOT TOUCHED, AND CONFIRMED NOT TOUCHED

Settlement was not started. No ledger file, no `ledger/flows.ts`, no `ledger/post.ts`, no settlement hold, no payout logic. OTP and signup are unchanged — nothing was added to the sign-up flow. The refund state machine, the late-payment policy, the webhook boundary, the refund execution ordering and the reconciliation detectors are all byte-identical. Nothing merged to `main`.

B

The data path, buyer to provider

```

buyer signs in (phone OTP)
└─ Better Auth  user.phoneNumber          verified by the OTP
  user.name = the phone string            placeholder, NOT NULL
└─ users.auth_user_id _____ the link
  users.display_name = 22790...@phone.afrinext.local  placeholder
  users.country_code = NULL                  no signup path writes it

checkout          createCheckout()          prices the order server-side
└─ order row      total_minor, currency

pay              payOrderAction()          posts orderId only
└─ initiatePayment()
  │ reads the ORDER for the amount          unchanged
  │ reads the BUYER for their details      NEW
  │   select u.country_code, a."phoneNumber"
  │   from users u
  │   left join "user" a on a.id = u.auth_user_id
  │   where u.id = actor.userId
  │ writes the payment row (initiated)      before the call
  └─ provider.createCharge({
      customer: { userId, phone },
      channel,
      metadata: { country, buyerCountry },
    })

```

Every field is read server-side from the buyer's own account. None is accepted from the caller, and that is the same rule the amount already follows: a field a caller can set is a field a caller can change. A buyer must not be able to tell the provider that somebody else's number is paying.

The phone comes from Better Auth's table because that is where sign-in verified it. Using the verified number matters — it is the one that answered an OTP, not one typed into a form.

PROVIDER FIELD	SOURCE	STATE
msisdn	user."phoneNumber"	SUPPLIED The verified sign-in number
country	users.country_code	SUPPLIED WHEN PRESENT No signup path writes it, so in practice absent
Ipay-Payment-Type	the caller's <code>channel</code>	SUPPLIED WHEN CHOSEN Never defaulted
customer_name	—	NOT AVAILABLE Afrinext has never collected one

Why no name is sent

Two columns look like they hold a name and neither does:

COLUMN	WHAT IT ACTUALLY HOLDS
users.display_name	<code>22790123456@phone.afrinext.local</code> — the synthetic address signup mints so Better Auth's NOT NULL email is satisfied
user.name	The phone string itself, written for the same reason

Passing either would be inventing a value for a provider-required field, which this milestone is explicitly not allowed to do. The second would additionally hand a payment provider an internal hostname. A test asserts that nothing matching `phone.afrinext.local` ever appears in the metadata.

There is also nowhere a buyer could supply a name: the application has no profile or account-settings screen at all.

Why the country travels twice

iPayMoney documents `country` only as « *le code du pays de la transaction* » and does not say whose country that is. That is question **K17**, and it is unanswered.

The order layer must send something or send nothing, and it holds two countries: the buyer's and the store's. It sends the **buyer's**, because the buyer is the payer — and it sends the same value a second time under `buyerCountry`, which states unambiguously what the value is.

This does not answer K17. When iPayMoney answers, the fix is to change which of our facts fills `country`; nobody has to reconstruct what the value once meant. The choice is recorded as assumption **A11**.

A test makes the buyer Nigerien and the seller Ivorian, so "the buyer's country was passed" is a real assertion rather than a coincidence of both being `NE`.

What happens when a field is missing

The adapter refuses, naming what is absent, and **no request is constructed**. Four tests assert the refusal and count the calls into the injected transport; the count is zero in every case.

MISSING	MESSAGE NAMES	REQUESTS BUILT
Phone	msisdn	0
Country	metadata.country	0
Channel	payment type	0
Name	customer_name	0

A DEFECT THIS MILESTONE HAD TO FIX TO MAKE "REFUSED SAFELY" TRUE

Before this change, a throw from `createCharge` left the payment row at `initiated` forever. `payments_one_live_per_order` means `initiatePayment` returns any live row instead of calling the provider — so every later attempt on that order returned the stuck row and did nothing. **The buyer would have clicked pay forever.**

It never showed under the mock, which does not throw. It would have shown on the first real iPayMoney charge. A refusal now closes the payment and the order, so the buyer gets a clear failure and can start a fresh checkout — the same outcome the existing "provider refuses at initiation" path already produced.

A throw is not a failed charge

Closing the payment on a refusal is safe only because the refusal is **classified**, not assumed. This is the approved Phase 3 correction applied one layer up: **unknown must never become failed**.

WHAT WAS THROWN	TREATED AS	PAYMENT
<code>ProviderNotConfiguredError</code> — the adapter declined to build a request, or the provider answered a documented refusal stating no payment was created	NO CHARGE EXISTS	failed , order closed
Transport failure at stage <code>not_transmitted</code> — DNS, connection refused, unreachable host	NO CHARGE EXISTS	failed , order closed
Timeout, connection reset, HTTP 5xx, unparseable answer	UNKNOWN	left <code>initiated</code> , order left <code>pending_payment</code>

A 5xx arrives *after* the request reached iPayMoney, so the charge may exist under our idempotency key. Marking it failed would assert that no money moved, which nobody can support. A test drives a real 502 through the adapter and asserts the payment stays `initiated` and the order stays open.

The adapter owns the classification of what its own errors mean; this layer honours it rather than re-deciding from an HTTP status. That is the same division the refund path already uses.

Security invariants, all preserved

INVARIANT	STATE
Provider verification — a webhook is confirmed only by a provider lookup	UNCHANGED
Idempotency — the key is derived, one live payment per order	UNCHANGED, and newly asserted
<code>transaction_id</code> protections	UNCHANGED
<code>unknown</code> / <code>in_doubt</code> classification	EXTENDED to charges, section E
Late-payment policy and the grace window	UNCHANGED · 31 tests green
<code>refund_due</code> as a terminal order state	UNCHANGED
Reconciliation and its detectors	UNCHANGED · clean, both probes firing
Row locking and deterministic lock order	UNCHANGED
<code>providerEventId</code> derivation and the replay index	UNCHANGED · 5 tests green
<code>statesChargeAmount = false</code> for <code>iPayMoney</code>	UNCHANGED
The amount is read from the order, never from a caller	UNCHANGED — and the new fields follow the same rule

Specifically, and by test: our own order amount is never copied into `ChargeStatus.amount`; an HTTP 5xx is never proof of failure; a webhook status is never authoritative without the provider lookup; and no customer refund API exists anywhere in the codebase.

Tests added, and the full results

Thirteen tests in one new file, all driving the real `createCheckout → initiatePayment` path against a real database with real users, real consent and a real published product.

#	WHAT IT PROVES
1	A buyer with a verified number reaches the provider with that exact msisdn
2	The buyer's country is passed — the seller is in a different country, so a substitution would fail here
3	The chosen channel is passed, and when none is chosen none is sent
4	No customer name is sent, and nothing matching <code>phone.afrinext.local</code> appears in the metadata
5	A buyer with no country sends no country, rather than a guess
6	An account with no verified number sends no phone, and still identifies the buyer by user id
7	The real adapter refuses naming <code>customer_name</code> , and no longer names the payer or the country
8–10	Missing channel, phone and country each refuse with zero requests constructed
11	A proven refusal closes the payment and the order — no row stuck at <code>initiated</code>
12	A 502 leaves the payment <code>initiated</code> and the order open — unknown must never become failed
13	Two initiations return one payment and call the provider once

Test 7 asserts the **current** truth. When a name becomes available it will fail, and that failure is the signal that the last field landed.

LOCAL PROVIDER-TRANSPORT TEST, NOT A SANDBOX TEST

Tests 7–12 use the **real iPayMoney adapter over an injected `fetch`** . Nothing reaches iPayMoney. **No sandbox credentials exist and no request has ever been made to iPayMoney from this repository.** `ipaymoney.integration.test.ts` remains the only file that talks to the real provider, and its 16 real tests remain skipped.

Results at 49c043f

CHECK	RESULT	AGAINST THE PREVIOUS BASELINE
pnpm lint	CLEAN	unchanged
pnpm typecheck	CLEAN	unchanged
pnpm test	491 PASSED · 16 SKIPPED (26 files)	was 478 / 16 / 25 files. +13 is exactly the new file; no existing count moved
pnpm build	COMPILED	unchanged
playwright	31 PASSED	unchanged
reconcile-check	CLEAN	drift 0 · 0 unbalanced · net XOF 0 · both probes firing
CI run #44 · 32488771572	SUCCESS	lint · typecheck · test, dependency scan, and browser — all three green
mutation matrix	24 / 26 KILLED	identical to the previous baseline — same 24 killed, the same 2 survivors, and every failure count unchanged

No Phase 3 test changed. The refund, late-payment, webhook and replay suites are untouched and green: 80 refund tests, 31 late-payment, 44 iPayMoney adapter, 5 replay, 46 orders.

One existing file changed shape without changing behaviour: createTestUser gained an optional phone that creates the Better Auth credential row real signup creates. Existing callers pass nothing and get exactly what they got before. The browser journey gained one assertion — that a real phone-OTP signup produces the identity link a charge depends on, which is the half no unit test can prove.

The matrix was re-run in full against 49c043f in the isolated worktree. R7 and R8 survive for the reason they always have: three independent mechanisms serialise a refund dispatch, so removing any one changes no behaviour. R26 removes all three at once and is killed. Because every individual failure count matches the previous run exactly, this milestone weakened nothing the matrix can see.

One, recorded as A11 in the assumptions register rather than left in code.

A11

Afrinext's checkout provisionally sends the **buyer's** country in `country`. The order layer holds two countries and must choose or send nothing; the buyer is the payer, so the buyer's country is sent.

It is provisional and it does **not** answer K17. The same value travels as `buyerCountry`, which names what it actually is, so answering K17 changes which fact fills `country` rather than requiring anyone to work out what was meant.

No other assumption was made. In particular: no default channel was chosen, no name was synthesised, no country was inferred from a phone prefix or from the store, and no provider behaviour was invented. Nothing was added to `support-questions.md` — the code raised no new question that K1–K18 did not already cover.

Remaining blockers

#	BLOCKER	WHOSE	STATE
S7	Checkout does not pass the buyer's details	Afrinext	MOSTLY CLOSED — phone, country and channel now flow; <code>customer_name</code> remains
S1	Phase 3 has not been approved	You	OPEN
S2	Settlement timing T+N undocumented — K16	iPayMoney	OPEN
S3	The amount is not verifiable from the provider — K8	iPayMoney	OPEN
S4	The 3 % fee's treatment on refund — K11	iPayMoney	OPEN
S5	No customer-refund execution to reverse — K1, K5	iPayMoney	OPEN
S6	No real payment has ever been made	Credentials	OPEN
S8	New. No buyer name is collected anywhere, so <code>customer_name</code> cannot be sent	Product decision	NEEDS YOU

Real sandbox testing is still blocked by credentials. Sixteen real-network tests stay skipped, gated on `IPAYMONEY_BASE_URL`, `IPAYMONEY_API_KEY` and an explicit `IPAYMONEY_ENVIRONMENT=sandbox`. A missing or mistyped environment skips rather than proceeding, and it can never fall back to Live: the variable has no default at all.

K1–K18 remain awaiting iPayMoney. None was modified and none was answered.

Decisions needed from you

1

Where does a buyer's name come from?

This is the last field, and it is a product question rather than a wiring one.

Three routes, and I am not taking any of them without you:

- **Ask at checkout.** One field on the payment step, stored on the order or the account. Touches no authentication and is the smallest change — but it adds friction exactly where a buyer is about to pay.
- **Ask once, after signup.** A profile step that fills `users.display_name` with a real name. Better data, and it needs a screen the application does not have.
- **Ask at signup.** Best data, and it changes the OTP flow — which I will not touch without explicit approval.

The same screen could collect the buyer's country, which would close S8 and make `country` reliably present at the same time.

2

Should checkout offer a payment channel?

iPayMoney documents `mobile` and `card` with **no default**, so the adapter refuses without one and nothing defaults it. Today no screen asks. The smallest change is a two-option control on the payment step; whether Afrinext offers cards at launch at all is your call, not an implementation detail.

3

Sandbox credentials.

Unchanged from the last report and still the thing that turns hypotheses into facts. Sixteen tests are written and waiting.

NOTHING HERE NEEDS RETROACTIVE APPROVAL

No approved state machine or business rule was changed. The one assumption made is A11, and it is provisional, documented, and reversible in one line. Settlement was not touched, nothing is merged to `main`, no sandbox readiness is claimed, and no provider behaviour was invented.